

- Secrets
- ABAP
- Ansible
- Apex
- AzureResourceManager
- C
- C#
- C++
- CloudFormation
- COBOL
- CSS
- Dart**
- Docker
- Flex
- Go
- HTML
- Java
- JavaScript
- JCL
- Kotlin
- Kubernetes
- Objective C
- PHP
- PL/I
- PL/SQL
- Python
- RPG
- Ruby
- Scala
- Swift
- Terraform
- Text
- TypeScript
- T-SQL
- VB.NET
- VB6
- XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules 115 Bug 15 Code Smell 100

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name... 🔍

Class names should comply with a naming convention	
Track uses of "TODO" tags	
Deprecated code should be removed	
Cyclomatic Complexity of functions should not be too high	
Constant names should comply with a naming convention	
Unnecessary nullable in final declaration should be removed	
Dependencies should be sorted	
Unused assignments should be removed	
Standard outputs should not be used directly to log anything	
Variables should not be initialized with "null"	
Files should end with a newline	

Variables should not be initialized with "null"

Analyze your code

Code Smell Minor clumsy

Why is this an issue?

More Info

In Dart, there’s no concept of "uninitialized memory". Everything must be initialized before use, otherwise a compile-time error is reported. In case of non-nullable type, it has to be explicitly initialized before use, and it can’t be initialized with `null`. This is guaranteed by compiler. In case of non-nullable variable, it will be set to `null` implicitly. In both cases there is no need to initialize a variable with `null`.

Exceptions

In case of `final` and `const` variables or members, they have to be initialized explicitly, so using `null` there won’t trigger this rule.

```
const int? x = null;
```

Noncompliant code example

```
void f() {
  int? x = null;
  g(x);
}
```

Compliant solution

```
void f() {
  int? x;
  g(x);
}
```

Available In:

sonarcloud

